CASO PRÁCTICO 2 PYTHON

DANIEL FERNÁNDEZ GUTIÉRREZ

Importación de librerías

In [305…
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Carga de Datos

In [306…
```
df_clientes = pd.read_excel('/content/Clientes_TiendaRopa.xlsx')
```

In [307…
```
df_productos = pd.read_excel('/content/Productos_TiendaRopa.xlsx')
```

In [308…
```
df_proveedores = pd.read_excel('/content/Proveedores_TiendaRopa.xlsx')
```

In [309…
```
df_ventas = pd.read_excel('/content/Ventas_TiendaRopa.xlsx')
```

In [310…
```
df_clientes.sample(5)
```

Out[310…

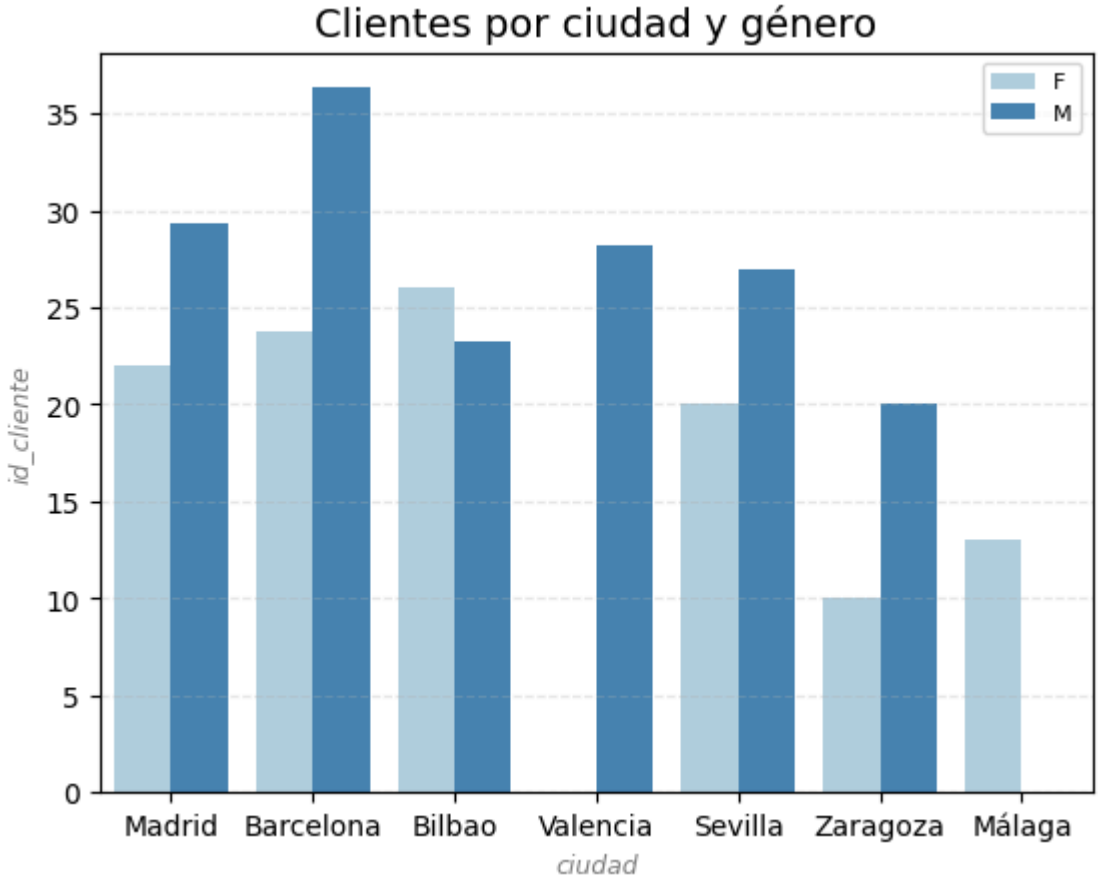
	id_cliente	nombre	email	telefono	fecha_registro	ciu
10	11	Teresa Hunt	t.hunt22@gmail.com	(+34 7123643219)	2023-01-07	Ma
6	7	Natalia Wilson	natalia_wilson2023@gmail.com	(+34 7445644899)	2023-11-07	Barce
1	2	Ana Simpson	ana.simpson82@hotmail.es	(+34 7445553219)	2023-10-14	Barce
39	40	Aarón Flores	aaron.flores56@gmail.com	(+34 7000643779)	2023-10-01	Se
34	35	Santiago Berry	s.berry77@outlook.es	(+34 7447773219)	2025-08-05	Vale

Gráfico de Barras Seaborn

Pregunta: ¿Cuál es la distribución de los clientes por ciudad y género?

Preparación: Vale su DataFrame original.

In [311…
```
sns.barplot(df_clientes, x='ciudad', y='id_cliente', hue='genero',palette='Blues
plt.title('Clientes por ciudad y género', fontsize=14, color='black')
plt.xlabel('ciudad', fontsize=9, fontstyle='italic', color='gray')
plt.ylabel('id_cliente', fontsize=9, fontstyle='italic', color='gray')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.legend(fontsize=8, loc='upper right')
plt.show()
```



Interpretación: En Barcelona se encuentra la mayor parte de la clientela. Valencia concentra exclusivamente clientes masculinos y Málaga clientes femeninos.

In [312…
```
df_ventas.sample(5)
```

Out[312…

	id_venta	fecha_venta	id_cliente	total_venta	metodo_pago
166	167	2024-08-24 03:42:41	33	1452.98	Transferencia
112	113	2025-03-20 22:46:42	36	400.79	Tarjeta
174	175	2024-09-25 14:12:58	26	370.11	Tarjeta
79	80	2025-03-24 14:41:41	25	319.83	Tarjeta
65	66	2024-10-03 04:05:42	35	357.94	Tarjeta

In [313…
```
df_ventas_var=df_ventas.copy()
```

In [314…
```
df_ventas_var['fecha_venta']=pd.to_datetime(df_ventas['fecha_venta'])
```

In [315…
```
df_ventas_var['año_venta']=df_ventas['fecha_venta'].dt.year
```

In [316…
```
df_ventas_var['mes_venta']=df_ventas['fecha_venta'].dt.month
```

In [317…
```
df_ventas_var.head()
```

Out[317…

	id_venta	fecha_venta	id_cliente	total_venta	metodo_pago	año_venta	mes_venta
0	1	2025-01-24 21:03:07	22	48.34	Efectivo	2025	1
1	2	2025-03-09 02:14:09	47	302.09	Tarjeta	2025	3
2	3	2025-01-15 02:34:16	4	387.07	Tarjeta	2025	1
3	4	2024-11-30 19:50:28	18	1019.27	Transferencia	2024	11
4	5	2024-10-25 00:17:15	40	443.48	Tarjeta	2024	10

Gráfico de torta

Pregunta: ¿Cómo se distribuyen las formas de pago de las ventas?

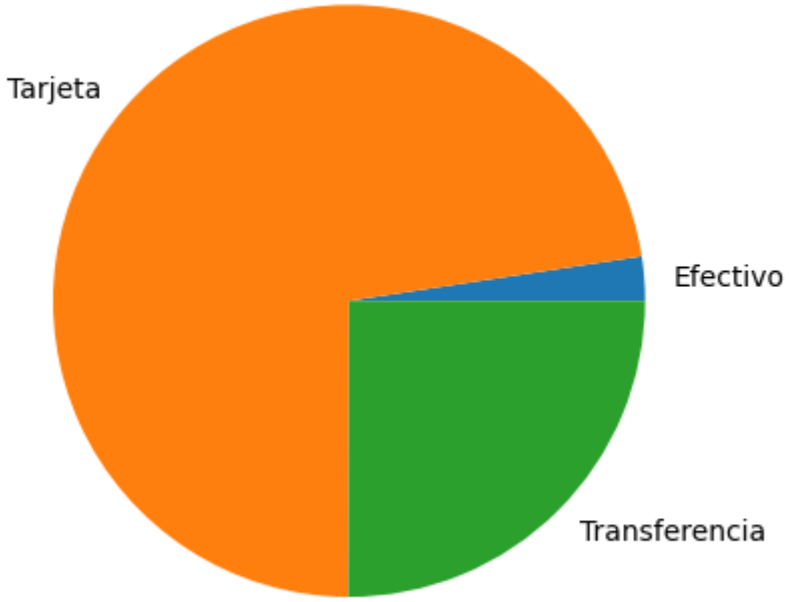
Preparación: Creamos dos nuevas variables (año_venta, mes_venta) y agrupamos las ventas totales según el método de pago elegido para poder crear el gráfico.

In [318…
```
df_ventas_metodo=df_ventas_var.groupby('metodo_pago').agg({'total_venta':'sum'})
df_ventas_metodo
```

Out[318…

metodo_pago	total_venta
Efectivo	2420.57
Tarjeta	72931.42
Transferencia	25173.28

In [319…
```
plt.pie(df_ventas_metodo['total_venta'], labels=df_ventas_metodo.index);
```



Interpretación: El pago con tarjeta representa la gran mayoría de las ventas totales, siguiéndole el pago por transferencia. Sin embargo, el pago en efectivo es casi inexistente.

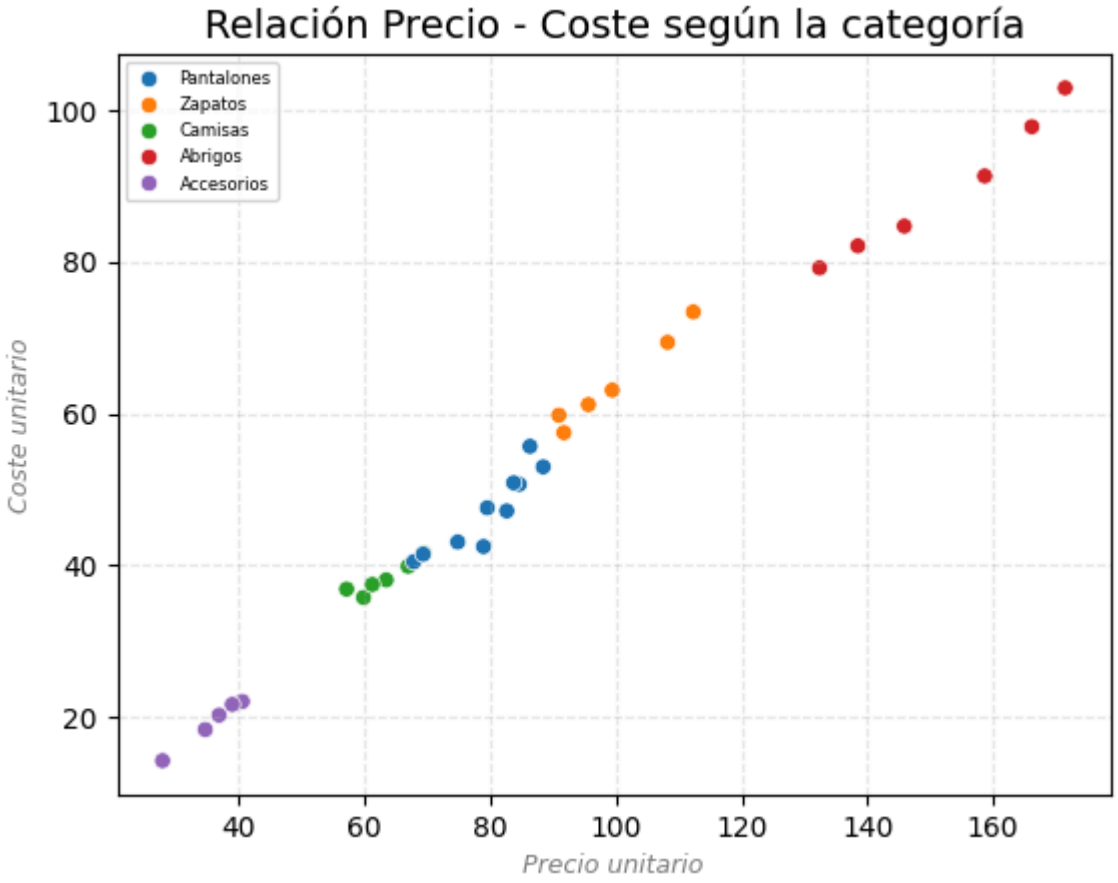
Gráfico de Puntos Seaborn

Pregunta: ¿Es positiva la relación entre Precio y Coste según su categoría?

Preparación: Vale su DataFrame original.

In [320…

```
sns.scatterplot(df_productos, x='precio_unitario', y='costo_unitario', hue='cate
plt.title('Relación Precio - Coste según la categoría',fontsize=14)
plt.xlabel('Precio unitario', fontsize=9, fontstyle='italic', color='gray')
plt.ylabel('Coste unitario', fontsize=9, fontstyle='italic', color='gray')
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.legend(fontsize=6, loc='upper left')
plt.grid(axis='x', linestyle='--', alpha=0.3)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```



Interpretación: La relación es positiva, a mayor coste unitario, mayor es el precio de venta, indicando la existencia de un margen de ganancia constante.

Gráfico de Barras Matplotlib

Pregunta: ¿Cuántos proveedores tiene cada país?

Preparación: Se agrupa el df por país y se cuenta el número de proveedores de cada uno.

In [321…

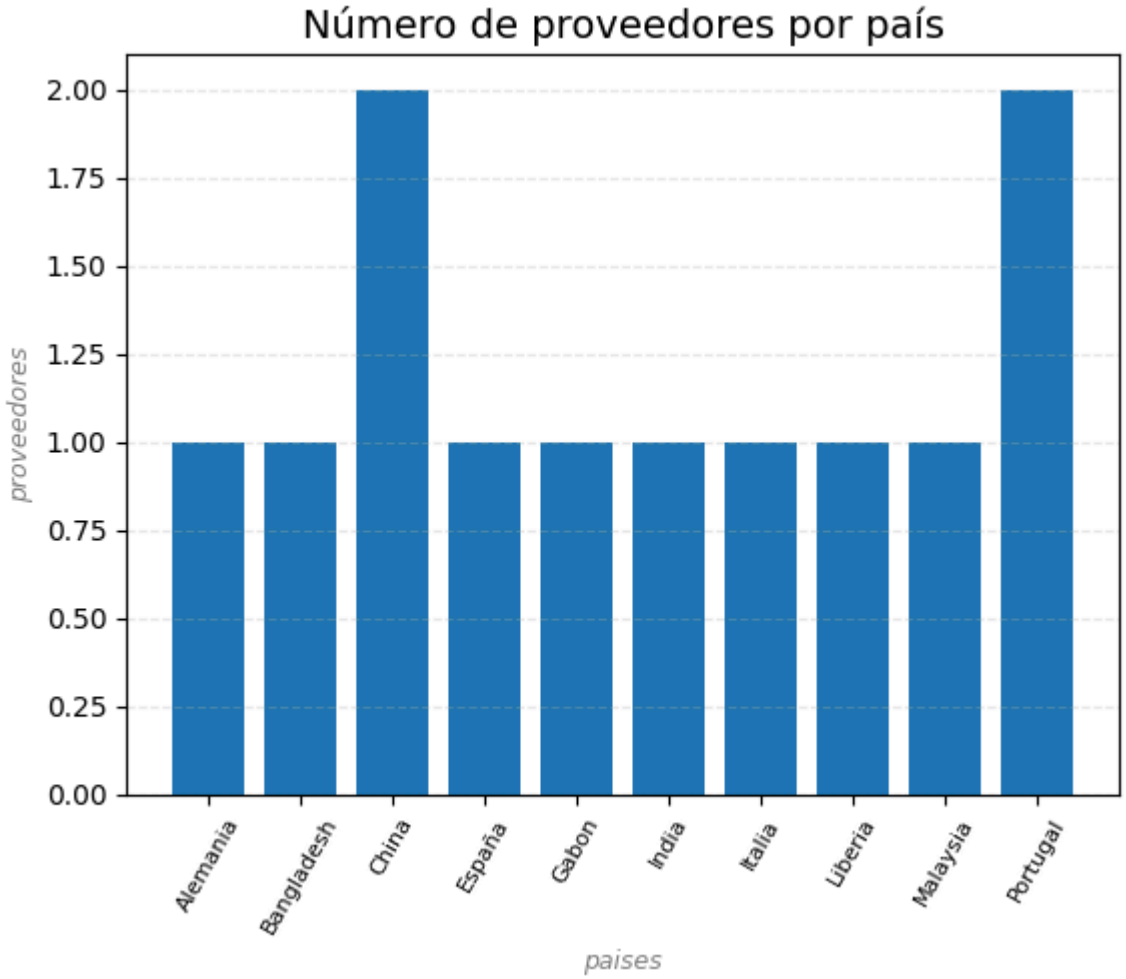
```
df_provpais=df_proveedores.groupby('pais').agg({'nombre_proveedor':'count'})
df_provpais
```

Out[321…

pais	nombre_proveedor
Alemania	1
Bangladesh	1
China	2
España	1
Gabon	1
India	1
Italia	1
Liberia	1
Malaysia	1
Portugal	2

In [322…

```
plt.bar(df_provpais.index, df_provpais['nombre_proveedor'])
plt.title('Número de proveedores por país', fontsize=14, color='black')
plt.xlabel('paises', fontsize=9, fontstyle='italic', color='gray')
plt.ylabel('proveedores', fontsize=9, fontstyle='italic', color='gray')
plt.xticks(rotation=60, fontsize=8)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```



Interpretación: La mayoría de países tiene un único proveedor, siendo China y Portugal los únicos con dos.

Preparación datos:

1. Unión de tablas ventas + clientes.
2. Comprobar si hay nulos.
3. Eliminarlos.
4. Pasar las columnas año y mes a número entero porque salían en decimales.
5. Agrupar el total de las ventas por año y mes.

In [323…
```
df_vc=df_clientes.merge(df_ventas_var, on='id_cliente', how='left')
```

In [324…
```
df_vc.isnull().sum()
```

Out[324...

	0
id_cliente	0
nombre	0
email	0
telefono	0
fecha_registro	0
ciudad	0
fecha_nacimiento	0
genero	0
id_venta	2
fecha_venta	2
total_venta	2
metodo_pago	2
año_venta	2
mes_venta	2

dtype: int64

In [325...
```
df_vc[df_vc['id_venta'].isnull()]
```

Out[325...

	id_cliente	nombre	email	telefono	fecha_registro	ciud
5	2	Ana Simpson	ana.simpson82@hotmail.es	(+34 7445553219)	2023-10-14	Barcel
25	9	Susana Brewer	susana.brewer_89@gmail.com	(+34 7442223989)	2025-06-02	Mad

In [326...
```
df_vc1=df_vc.dropna()
```

In [327...
```
df_vc1.duplicated()
```

Out[327…

	0
0	False
1	False
2	False
3	False
4	False
…	…
197	False
198	False
199	False
200	False
201	False

200 rows × 1 columns

dtype: bool

In [328…

```
df_vc1['año_venta']=df_vc1['año_venta'].astype(int)
df_vc1['mes_venta']=df_vc1['mes_venta'].astype(int)
```

```
/tmp/ipykernel_1994/3104632461.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_vc1['año_venta']=df_vc1['año_venta'].astype(int)
/tmp/ipykernel_1994/3104632461.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_vc1['mes_venta']=df_vc1['mes_venta'].astype(int)
```

In [329…

```
df_vc1.sample(3)
```

Out[329…

	id_cliente	nombre	email	telefono	fecha_registro	ciuda
135	32	Jessica Howell	jessica.howell57@gmail.com	(+34 7411143449)	2023-07-19	Bilba
56	17	Amy Simón	amy.simon67@gmail.com	(+34 7465443329)	2025-04-07	Madri
145	35	Santiago Berry	s.berry77@outlook.es	(+34 7447773219)	2025-08-05	Valenc

In [330…
```
df_eje=df_vc1.groupby(['año_venta','mes_venta']).agg({'total_venta':'sum'}).rese
df_eje
```

Out[330…

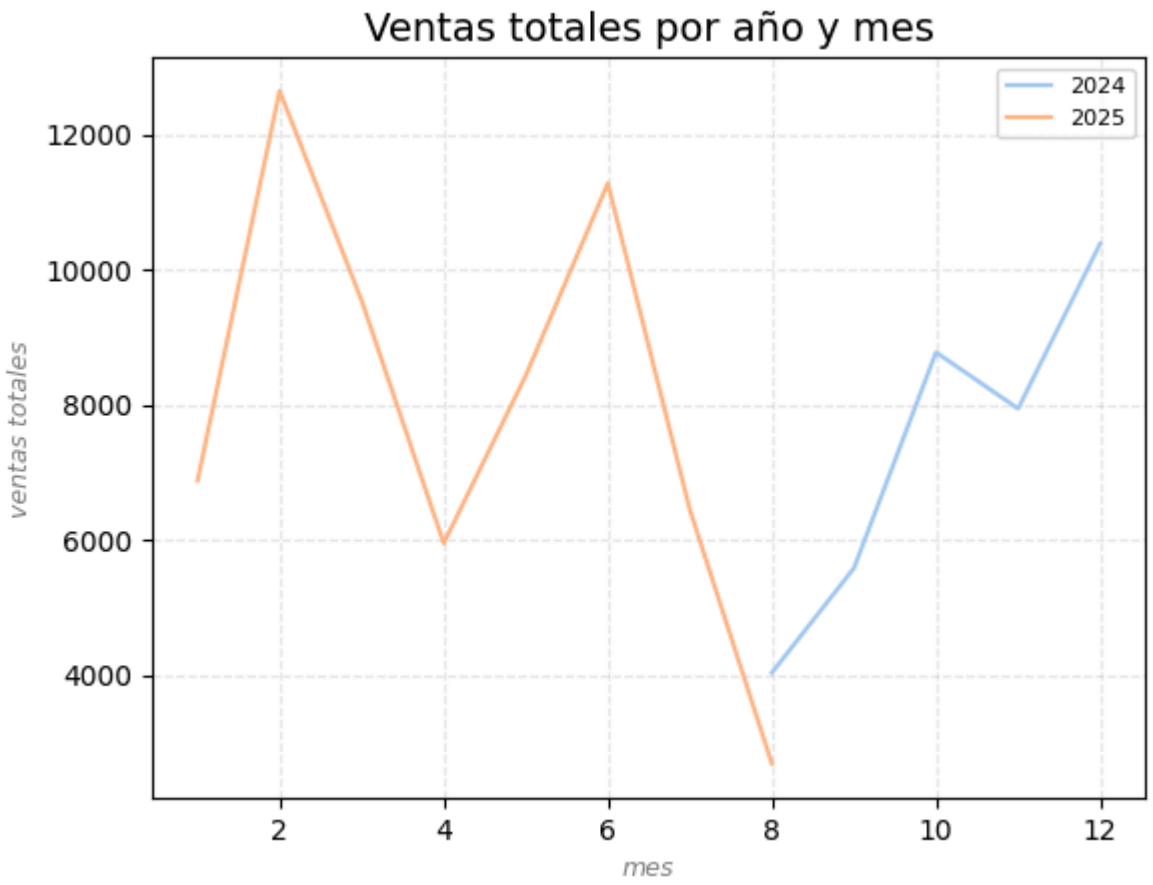
	año_venta	mes_venta	total_venta
0	2024	8	4031.06
1	2024	9	5579.52
2	2024	10	8771.39
3	2024	11	7939.23
4	2024	12	10379.47
5	2025	1	6873.76
6	2025	2	12639.58
7	2025	3	9537.76
8	2025	4	5946.44
9	2025	5	8428.62
10	2025	6	11272.85
11	2025	7	6440.94
12	2025	8	2684.65

Gráfico de Líneas en Seaborn

Pregunta: ¿Cuál es la evolución de las ventas a lo largo de estos dos años?

Preparación: Limpiar los datos, ya que había dos filas con valores nulos. Por último, agrupar el total de las ventas por año y mes.

In [331…
```
sns.lineplot(df_eje, x='mes_venta', y='total_venta', hue='año_venta', palette='p
plt.title('Ventas totales por año y mes', fontsize=14, color='black')
plt.xlabel('mes', fontsize=9, fontstyle='italic', color='gray')
plt.ylabel('ventas totales', fontsize=9, fontstyle='italic', color='gray')
plt.grid(True, linestyle='--', alpha=0.3)
plt.legend(fontsize=8, loc='upper right')
plt.show()
```



Interpretación: Las ventas comienzan en agosto de 2024, observándose un incremento constante hasta febrero de 2025, mes en el cual hay un grave descenso de las ventas. En abril remontan para marcar su mínimo en agosto de 2025.

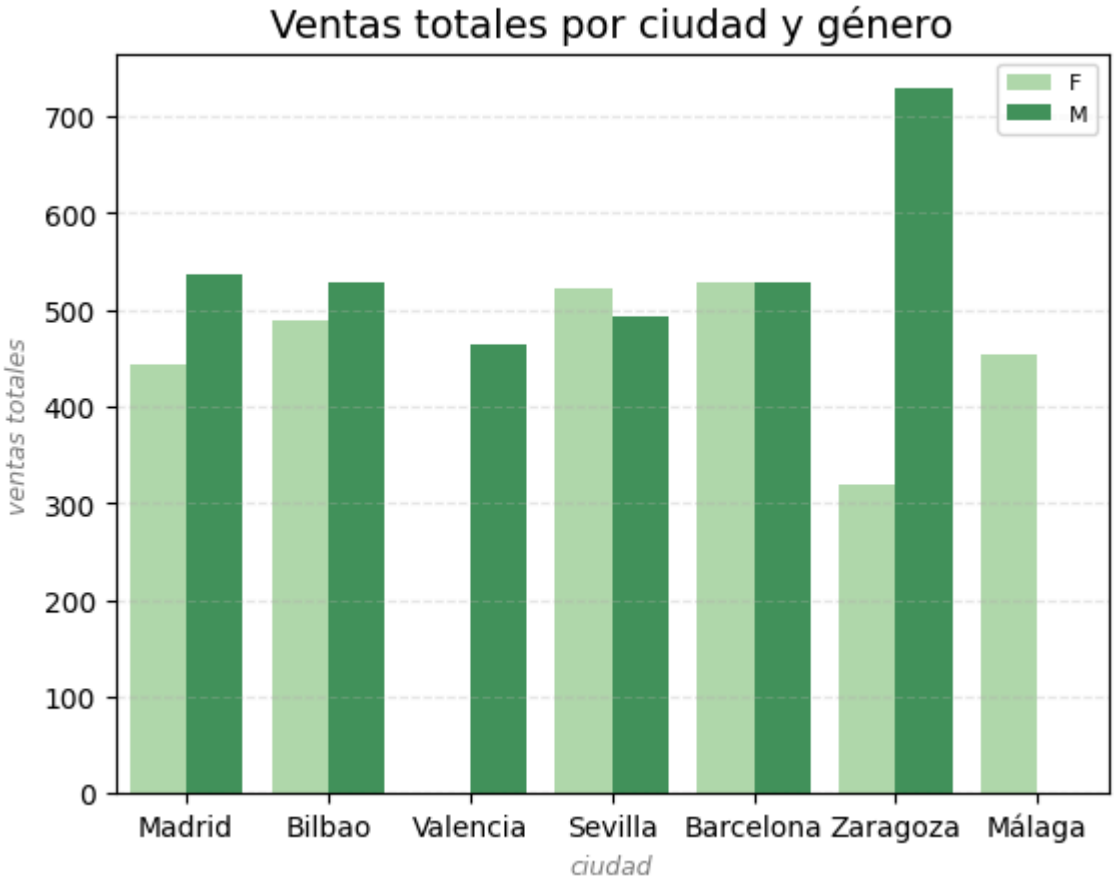
Gráfico de Barras en Seaborn

Pregunta: ¿Cuál es la distribución de las ventas totales por ciudad y género?

Preparación: Tras unir y limpiar las tablas, obtenemos un nuevo DataFrame llamado df_vc_limpio, resultante de unir y limpiar la tabla ventas con la de clientes.

In [332…

```
sns.barplot(df_vc1, x='ciudad', y='total_venta', hue='genero', palette='Greens',
plt.title('Ventas totales por ciudad y género', fontsize=14, color='black')
plt.xlabel('ciudad', fontsize=9, fontstyle='italic', color='gray')
plt.ylabel('ventas totales', fontsize=9, fontstyle='italic', color='gray')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.legend(fontsize=8, loc='upper right')
plt.show()
```



Interpretación: Las ventas según el género y la ciudad están distribuidas equitativamente, salvo Zaragoza, en la cual la mayoría de las ventas pertenecen a hombres. En Valencia y Málaga, como comenté antes, habría que captar al género ausente.

Preparación datos: Unimos las tablas proveedores + productos.

In [333…
```
df_proveedores.sample(2)
```

Out[333…

	id_proveedor	nombre_proveedor	telefono	email	
3	4	Pereira & Madeira Lda.	(+351) 6217896824	zstout@pereiramadeira.pt	Port
5	6	Rahman & Sons Textiles	(+880) 6788196846	andrew.rahman@rahmansons.bd	Bangla

In [334…
```
df_productos.sample(2)
```

Out[334…

	id_producto	nombre_producto	categoria	precio_unitario	costo_unitario	id_prove
24	25	Capital Brazalete	Accesorios	34.6	18.4	
5	6	Total Abrigo	Abrigos	171.5	102.9	

In [335...
```
df_pp=df_productos.merge(df_proveedores, on='id_proveedor', how='left')
```

In [336...
```
df_pp[df_pp.isnull().any(axis=1)] #Comprobamos que no hay valores nulos.
```

Out[336...

id_producto	nombre_producto	categoria	precio_unitario	costo_unitario	id_proveedo

In [337...
```
df_pp[df_pp.duplicated()] #Comprobamos que no hay valores duplicados.
```

Out[337...

id_producto	nombre_producto	categoria	precio_unitario	costo_unitario	id_proveedo

In [338...
```
df_pp.sample(2)
```

Out[338...

	id_producto	nombre_producto	categoria	precio_unitario	costo_unitario	id_provee
7	8	Identify Deportivas	Zapatos	108.2	69.4	
18	19	Charge Botass	Zapatos	112.3	73.4	

Gráfico de torta

Pregunta: ¿Existe riesgo de dependencia de proveedores en la empresa?

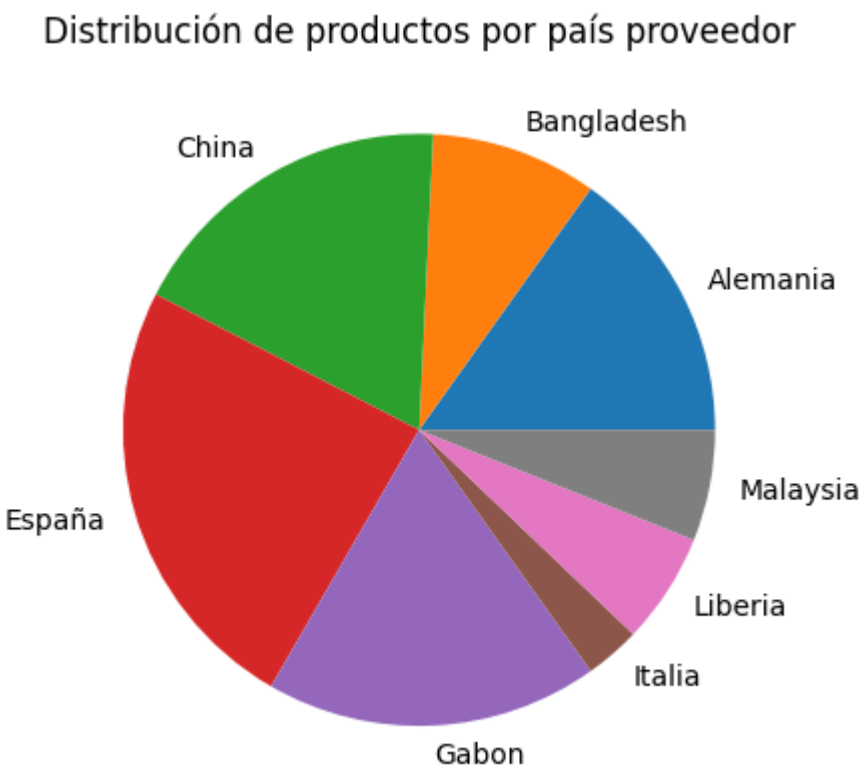
Preparación: Unión de las tablas productos y proveedores. Para crear el gráfico, es necesario agrupar por país y nombre de producto.

In [339...
```
df_pp_pais= df_pp.groupby('pais').agg({'nombre_producto':'count'})
df_pp_pais
```

Out[339...

pais	nombre_producto
Alemania	5
Bangladesh	3
China	6
España	8
Gabon	6
Italia	1
Liberia	2
Malaysia	2

In [340…
```
plt.pie(df_pp_pais['nombre_producto'], labels=df_pp_pais.index)
plt.title('Distribución de productos por país proveedor')
plt.show()
```



Interpretación: El gráfico refleja una diversificación en los países proveedores. Ninguno de ellos supera el 50%, por lo que no hay evidencia de una dependencia absoluta. Si uno de los países sufriera problemas de logística, podría solucionarse con el respaldo de los demás países.

Modelos Predictivos

Preparación de los datos: Añadimos la columna edad restando año actual menos la columna fecha de nacimiento.

In [341…
```
df_vc1.sample(3)
```

Out[341…

	id_cliente	nombre	email	telefono	fecha_registro	
3	1	María Bell	maria.bell23@gmail.com	(+34 7445643219)	2024-06-18	I
38	12	Nathan Velázquez	nathan.velazquez@hotmail.com	(+34 7456643349)	2024-11-05	V
161	39	Gregorio Armstrong	greg.armstrong07@outlook.es	(+34 7445989277)	2022-12-18	I

In [342...
```
df_vc1.copy()
df_vc1['edad'] = 2026-pd.to_datetime(df_vc1['fecha_nacimiento']).dt.year
df_vc1.sample(2)
```

```
/tmp/ipykernel_1994/441996456.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_vc1['edad'] = 2026-pd.to_datetime(df_vc1['fecha_nacimiento']).dt.year
```

Out[342...

	id_cliente	nombre	email	telefono	fecha_registro	ciud
149	36	Jonathan Green	jonathan.green65@gmail.com	(+34 7098643339)	2025-03-24	Mad
54	17	Amy Simón	amy.simon67@gmail.com	(+34 7465443329)	2025-04-07	Mad

Importación de Librerías

In [343...
```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import mean_absolute_percentage_error, mean_absolute_error,

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
```

Preparación de los datos: Transformar las columnas 'ciudad_enc', 'genero_enc' y 'pago_enc' en números para poder tratarlos.

In [344...
```
df_vc1 = df_vc1.copy()
le = LabelEncoder()
df_vc1["ciudad_enc"] = le.fit_transform(df_vc1["ciudad"])
df_vc1["genero_enc"] = le.fit_transform(df_vc1["genero"])
df_vc1["pago_enc"] = le.fit_transform(df_vc1["metodo_pago"])
```

In [372...
```
df_vc1.sample(3)
```

Out[372...

	id_cliente	nombre	email	telefono	fecha_registro	
168	41	Cristóbal Schmitt	c.schmitt59@correo.com	(+34 7443323219)	2022-12-08	Ba
163	39	Gregorio Armstrong	greg.armstrong07@outlook.es	(+34 7445989277)	2022-12-18	
136	33	Elías Hernández	elias.hernandez61@correo.com	(+34 7222663219)	2025-01-28	

Separación Variable Objetivo (y) y Variables Explicativas (X)

In [345...

```
y=df_vc1['total_venta']
y
```

Out[345...

	total_venta
0	636.12
1	256.96
2	591.08
3	575.17
4	302.97
...	...
197	701.10
198	688.34
199	130.26
200	175.73
201	759.51

200 rows × 1 columns

dtype: float64

In [346...

```
X=df_vc1[['edad','ciudad_enc','genero_enc','pago_enc']]
X
```

Out[346...

	edad	ciudad_enc	genero_enc	pago_enc
0	31	2	0	1
1	31	2	0	1
2	31	2	0	1
3	31	2	0	1
4	31	2	0	1
...	...	...	...	...
197	53	2	1	1
198	28	0	0	1
199	28	0	0	1
200	28	0	0	1
201	28	0	0	1

200 rows × 4 columns

Separación en Train y Test

In [347...
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [348...
```
X_train.shape
```

Out[348... (160, 4)

In [349...
```
X_test.shape
```

Out[349... (40, 4)

In [350...
```
y_train.shape
```

Out[350... (160,)

In [351...
```
y_test.shape
```

Out[351... (40,)

Ajuste del modelo sobre el conjunto de Train

In [352...
```
modelo_regresion = LinearRegression()
```

In [353...
```
modelo_regresion.fit(X_train,y_train)
```

Out[353...

▾ LinearRegression

```
LinearRegression()
```

Predicción con el modelo sobre el conjunto de Test

In [354…
```
modelo_regresion.predict(X_test)
```

Out[354…
```
array([ 551.55035287,   98.93707059,  942.68251131,  426.22360432,
        580.89280199,  570.82527859,  446.35865112,  420.20275679,
        455.81998296,  574.87195446,  529.4411344 ,  570.82527859,
        495.22995548,  561.11001724,  506.82405692,  449.79913543,
        879.18914866,  450.6592565 ,  551.55035287,  538.04234516,
        490.57708805, 1022.67677234,  490.57708805,  542.18735357,
        446.35865112,  506.82405692,  472.41621291,  514.21288455,
        117.60580475,  524.88659951,  538.04234516,  877.4689065 ,
        506.82405692,  561.11001724,  485.67029111,  426.22360432,
        450.6592565 ,  562.57632988,  538.04234516,  542.94914211])
```

Medición del Modelo automática

In [355…
```
mean_absolute_error(y_test, modelo_regresion.predict(X_test)) #MAE
```

Out[355…
```
201.52186158757652
```

MAE: En promedio, el modelo se equivoca en 201 unidades arriba o abajo.

In [356…
```
mean_absolute_percentage_error(y_test, modelo_regresion.predict(X_test)) #MAPE
```

Out[356…
```
0.9503416520778787
```

MAPE: El error medio es del 95% respecto al valor real. Este porcentaje es muy alto, indica predicciones bastante alejadas del modelo.

In [357…
```
mean_squared_error(y_test, modelo_regresion.predict(X_test)) #MSE
```

Out[357…
```
63296.772820981205
```

MSE: Error cuadrático medio. Penaliza los errores grandes, Útil para comparar modelos, pero difícil de interpretar solo.

In [358…
```
root_mean_squared_error(y_test, modelo_regresion.predict(X_test)) #RMSE
```

Out[358…
```
251.58849898391858
```

RMSE: Raíz del MSE. El modelo se desvía aproximadamente 251 unidades de media.

In [359…
```
r2_score(y_test, modelo_regresion.predict(X_test)) #R²
```

Out[359…
```
0.10861525508739434
```

R^2: El modelo solo explica el 10.9% de la varianza de los datos. Es muy bajo porque el modelo no encuentra patrones.

Síntesis: Las variables explicativas no pueden predecir las ventas totales porque probablemente tienen poca relación directa con el importe vendido.

Comparación con Decision Tree: Como el modelo ha fallado bastante, voy a compararlo con el Decision Tree

In [360…
```
df_vc1.head()
```

Out[360…

	id_cliente	nombre	email	telefono	fecha_registro	ciudad	fech
0	1	María Bell	maria.bell23@gmail.com	(+34 7445643219)	2024-06-18	Madrid	
1	1	María Bell	maria.bell23@gmail.com	(+34 7445643219)	2024-06-18	Madrid	
2	1	María Bell	maria.bell23@gmail.com	(+34 7445643219)	2024-06-18	Madrid	
3	1	María Bell	maria.bell23@gmail.com	(+34 7445643219)	2024-06-18	Madrid	
4	1	María Bell	maria.bell23@gmail.com	(+34 7445643219)	2024-06-18	Madrid	

Queremos que el modelo aprenda a responder: Dado un cliente con cierta edad, ciudad, género y método de pago... ¿su compra será baja, media o alta?

Preparación datos: Transformamos el total venta (número) en nivel venta (texto) para poder tratarlos en el modelo. Dividimos el total venta en tres partes iguales y le asignamos las etiquetas: bajo, medio, alto.

In [361…
```
df_vc1['nivel_venta'] = pd.cut(
    df_vc1['total_venta'],
    bins=3,
    labels=['bajo', 'medio', 'alto'])
```

In [362…
```
X = df_vc1[['edad','ciudad_enc','genero_enc','pago_enc','año_venta', 'mes_venta'
y= df_vc1['nivel_venta']
```

Separación en Train y Test

In [363…
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

Ajuste del modelo sobre el conjunto de Train

In [364…
```
modelo_decision = DecisionTreeClassifier()
```

In [365…
```
modelo_decision.fit(X_train, y_train)
```

Out[365…
```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

Predicción con el modelo sobre el conjunto de Test

In [366…
```
modelo_decision.predict(X_test)
```

Out[366…
```
array(['bajo', 'bajo', 'medio', 'bajo', 'bajo', 'medio', 'medio', 'medio',
       'medio', 'bajo', 'bajo', 'medio', 'bajo', 'bajo', 'medio', 'bajo',
       'medio', 'bajo', 'bajo', 'bajo', 'bajo', 'alto', 'bajo', 'bajo',
       'bajo', 'medio', 'medio', 'bajo', 'bajo', 'bajo', 'bajo', 'medio',
       'medio', 'bajo', 'bajo', 'medio', 'bajo', 'bajo', 'medio', 'medio'],
      dtype=object)
```

Medición del Modelo automática

In [367…
```
accuracy_score(y_test, modelo_decision.predict(X_test))
```

Out[367…
```
0.625
```

Accuracy: Acierta el 65% de los casos. No es alta pero acierta más que el modelo anterior.

In [368…
```
precision_score(y_test, modelo_decision.predict(X_test) , average='weighted')
```

Out[368…
```
0.7466666666666667
```

Precision: Cuando predice una clase, acierta el 75.4% de las veces.

In [369…
```
recall_score(y_test, modelo_decision.predict(X_test) , average='weighted')
```

Out[369…
```
0.625
```

Recall: De todos los casos reales de cada clase, detecta el 65%.

In [370…
```
confusion_matrix(y_test, modelo_decision.predict(X_test))
```

Out[370…
```
array([[ 0,  0,  1],
       [ 1, 21, 10],
       [ 0,  3,  4]])
```

Matriz de Confusión: El nivel de venta medio (fila 2) es el que mejor predice (22 aciertos). Los otros niveles los confunde bastante debido a que hay un desbalanceo de clases (la clase 2 tiene muchos más ejemplos que las otras dos, por lo que el modelo aprende casi solo de ella).

In [371…
```
f1_score(y_test, modelo_decision.predict(X_test) , average='weighted')
```

Out[371…
```
0.6636363636363637
```

F1: Media entre precision y recall. Confirma el rendimiento moderado.

Síntesis final: Se han entrenado dos modelos sobre datos de ventas. La regresión lineal fue prácticamente inútil con un R^2 de 0.10, debido a que las variables elegidas tienen poca relación con el importe vendido. El árbol de decisión funcionó algo mejor con un 65% de exactitud, aunque sufre desbalanceo de clases, lo que le impide predecir bien las categorías minoritarias.